

Fase 1: identidad del proyecto

Renombre del paquete Java, prefijo de rutas /api/local, CORS de dos frontends y derivacion de apiLocal en Angular

Paquete raiz: com.erikjarquin.ventas -> com.erikjarquin.compras (164 archivos)

12 controllers movidos a /api/local/... (el conteo de endpoints quedo en 59)

Rutas publicas: /api/local/auth/** y /api/uploads/** (esta ultima fuera de /api/local a proposito)

CORS de desarrollo: localhost:4200 (Angular POS) + localhost:3000 (Next.js tienda)

Frontend: environment.apiLocal = \${api}/local, los 12 servicios HTTP migrados

Sin cambios de esquema en ESTA fase: al cierre habia 1 migracion (V1); hoy hay 2 (V1+V2)

Verificacion: 153 tests en verde (el unico @SpringBootTest exige BD real)

Indice

1. Que es la Fase 1 y por que se hizo antes de cualquier funcionalidad
2. Renombre del paquete Java: de 'ventas' a 'compras'
3. Por que 164 archivos y no un 'Find & Replace' a mano
4. El prefijo /api/local: la decision de arquitectura mas importante
5. SecurityConfig: que rutas son publicas y por que
6. CORS: por que hay que declarar dos origenes
7. El lado Angular: api vs apiLocal
8. Tablas de base de datos, PK y FK: por que no cambio nada
9. Endpoints: inventario completo
10. Glosario de anotaciones y conceptos
11. Verificacion: como se comprobo que nada se rompio
12. Trampas y errores que se cometieron en el camino

1. Que es la Fase 1 y por que se hizo antes de nada

El proyecto nacio como un POS (punto de venta) de una tienda de abarrotes: productos con codigo de barras y SKU, ventas, caja, proveedores. El plan (docs/PLAN.md) es transformarlo en una **tienda de comida**, con dos aplicaciones distintas frontalizando el mismo backend.

La tentacion es empezar por lo visible: crear la entidad Platillo o la tabla Pedido. Fue un error de orden. La Fase 1 no agrega ni una sola funcionalidad de comida: **solo renombra y ordena lo que ya existe**, para que las fases siguientes se escriban sobre una base coherente.

Por que esto va primero

Renombrar paquetes y rutas mas adelante significa editar 164 archivos otra vez, con el riesgo de que un replace a medias deje un import apuntando al paquete viejo y el error aparezca semanas despues, cuando ya no se correlaciona con el cambio que lo provoco. Renombrar primero hace que todo lo demas salga limpio.

El nombre 'ventas' NO se borro del dominio

Un punto que genera confusion: se renombro el **paquete** y el **artifactId**, pero **no** la palabra venta/ventas en el dominio. Venta / Sale sigue siendo un concepto valido y distinto de Pedido (el pedido online). Renombrar la entidad de ventas a 'pedidos' habria mezclado dos cosas distintas: la venta de mostrador y el pedido del cliente por web.

Que se renombro y que no

```
// Paquete: identidad del proyecto -> SI cambio
com.erikjarquin.ventas.model.entity.SaleEntity // antes
com.erikjarquin.compras.model.entity.SaleEntity // ahora (misma clase)

// Concepto de dominio: la venta de mostrador -> NO cambio
// y no debe confundirse con el Pedido online de la Fase 6
```

2. Renombre del paquete Java: de 'ventas' a 'compras'

El paquete raiz paso de com.erikjarquin.ventas a com.erikjarquin.compras. El nombre del paquete es el **namespace** de Java: lo que va antes de cada clase. Renombrarlo cambia la ruta de archivos en disco y obliga a actualizar todos los import.

Elemento	Antes	Despues
Paquete raiz	com.erikjarquin.ventas	com.erikjarquin.compras
Clase main	VentasApplication	ComprasApplication
Test de contexto	VentasApplicationTests	ComprasApplicationTests
pom.xml artifactId	ventas	compras
pom.xml name	(vacio)	compras
pom.xml description	(vacio)	API de compras, ventas e inventario
spring.application.name	compras	compras (ya era, no cambio)
Comment del Dockerfile	Ventas-Backend	Compras-Backend

La clase main, codigo real

```
src/main/java/com/erikjarquin/compras/ComprasApplication.java
package com.erikjarquin.compras;

/**
 * Clase principal de Compras-Backend (Spring Boot).
 *
 * <p>{@code @EnableScheduling} activa las tareas programadas, como
 * {@code PaymentMonitorJob}, que cada 5 minutos resuelve pagos con tarjeta que
 * quedaron PENDING. El escaneo de componentes (component-scan) cubre
 * automáticamente los paquetes: config, controller, service, repository, model,
 * mapper, exceptions y jobs.
 */
@SpringBootApplication
@EnableScheduling
public class ComprasApplication {

    public static void main(String[] args) {
        SpringApplication.run(ComprasApplication.class, args);
    }
}
```

El nombre de la clase DEBE coincidir con el nombre del archivo (public class ComprasApplication dentro de ComprasApplication.java). Java no compila si no. Ese es el unico motivo por el que el renombre no puede ser solo un cambio de carpeta.

3. Por que 164 archivos y no un 'Find & Replace' a mano

El renombre se hizo con git mv, no con mv ni con copiar y borrar. La diferencia no es cosmetica: **la deteccion de renombre de git** es lo que hace que el historial muestre 'archivo renombrado' en vez de 'archivo borrado (99 lineas) + archivo nuevo (99 lineas)'. Sin eso, un cambio de 164 archivos se convierte en 328 entradas en el log y nadie puede saber despues que fue un renombre.

El comando, con git mv y no mv

```
# Renombrar conservando el historial (desde la raiz del repo)
git mv src/main/java/com/erikjarquin/ventas \
      src/main/java/com/erikjarquin/compras
git mv src/test/java/com/erikjarquin/ventas \
      src/test/java/com/erikjarquin/compras

# Despues, sobre los archivos ya movidos: cambiar el namespace en las
# declaraciones 'package' y en todos los 'import'.
# Total: 164 archivos .java (main + test)
```

Trampa: el orden de las dos operaciones

El orden importa: primero git mv (mueve) y despues el replace de texto. Si se hace el replace antes de mover, git ve 'un archivo borrado y otro nuevo' porque el contenido Y la ruta cambiaron a la vez, y la deteccion de renombre se pierde. git compara similitud de contenido entre pares de archivos; si ya no son pares, no hay nada que comparar.

El replace de texto se limito a dos patrones exactos: la linea package com.erikjarquin.ventas y cualquier com.erikjarquin.ventas. dentro de un import. **Nunca** la palabra 'venta' suelta, porque esa es la palabra de dominio que se decide conservar.

4. El prefijo /api/local: la decision de arquitectura mas importante

Los 12 controllers pasaron de /api/<recurso> a /api/local/<recurso>. Parece cosmetico, pero es la decision que habilita todo el resto del plan.

El proyecto va a tener **dos frontends distintos** consumiendo el mismo backend: el POS de mostrador (Angular, para empleados) y la tienda online (Next.js, para clientes). Si ambos usaran /api/products, no habria forma de distinguir un endpoint para 'el cajero, devuelve margenes' de uno para 'el cliente, devuelve el catalogo'. Prefijarlos hace que la separacion sea visible desde la URL y no desde dentro del codigo.

SaleController.java - el cambio es de una linea

```
@RestController
@RequestMapping("/api/local/sales") // <- antes era @RequestMapping("/api/sales")
public class SaleController {
```

El mapa de los tres prefijos

Prefijo	Consumidor	Estado	Reglas
/api/local/**	Angular POS (empleados)	VIGENTE (Fase 1)	Requiere JWT + permiso por endpoint. 12 controllers.
/api/tienda/**	Next.js (clientes)	Fase 6, NO EXISTE	Publico o con registro de cliente. Nace en la Fase 3.
/api/uploads/**	Los dos frontends	VIGENTE (publico)	Sirve imagenes de productos/platillos. FUERA de /api/local a proposito.

Por que /api/uploads NO se movio a /api/local

/api/uploads/** quedo deliberadamente FUERA de /api/local y quedo publico. Lo tipico es meterse todo bajo el prefijo del backend, pero las imagenes de platillos las tiene que poder cargar el tag `img src` del storefront publico. Una imagen que necesita un header Authorization no se renderiza en un navegador. Cuando la Fase 3 construya el catalogo, las imagenes de platillo van a ser las MISMAS que las de producto: por eso el endpoint tiene que ser publico y compartido desde ahora.

5. SecurityConfig: que rutas son publicas y por que

SecurityConfig.java - la lista de rutas publicas (el corazon de la configuracion)

```
*/
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())
        .cors(Customizer.withDefaults())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()
            .requestMatchers("/api/local/auth/**").permitAll()
            .requestMatchers("/api/uploads/**").permitAll()
            .anyRequest().authenticated())
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

Los matchers con permitAll son la lista blanca publica. Todo lo demas exige autentificacion. El orden importa en Spring Security: se evalua de arriba hacia abajo y gana la primera regla que coincide, así que permitAll debe ir ANTES de anyRequest.

Como se autoriza en la practica

En el controlador el permiso se chequea con @PreAuthorize sobre el METODO, no en la clase. Es Spring Security method-level security: sin la anotacion a nivel de metodo el framework ni siquiera evalua la expresion. Y el string dentro de hasAuthority() debe coincidir EXACTAMENTE con el nombre del enum PermissionName: si no coincide, el permiso no existe y el acceso se deniega con 403.

6. CORS: por que hay que declarar dos origenes

CORS es una proteccion del NAVEGADOR, no del servidor: un navegador moderno no deja que el JavaScript de localhost:3000 llame a localhost:8081 si el servidor no lo autoriza explicitamente. El backend tiene que decir 'acepto peticiones de ambos origenes'.

El default de desarrollo

```
# application.yaml - Fase 1
cors:
  allowed-origins: ${CORS_ALLOWED_ORIGINS:http://localhost:4200,http://localhost:3000}

# Valores por defecto, en orden de lectura:
#   localhost:4200 = POS Angular, el que ya existia
#   localhost:3000 = tienda Next.js, se agrego AHORA aunque todavia no exista,
#                   para no olvidarlo en la Fase 6
```

Entorno	Valor de CORS_ALLOWED_ORIGINS
Local	http://localhost:4200,http://localhost:3000 (default del yaml)
Produccion	https://<app-angular>.netlify.app,https://<tienda>.vercel.app

Por que se centralizo en SecurityConfig

El CORS se configuro UNA SOLA VEZ, de forma global en SecurityConfig, leyendo la propiedad. Antes, cada controller tenia su propio @CrossOrigin. Con 12 controllers (y mas en las fases siguientes) eso son 12 lugares que mantener y la probabilidad de que uno se quede atras. Ademas @CrossOrigin a nivel de clase es un problema conocido de Spring: se combina mal con los handlers de error.

7. El lado Angular: api vs apiLocal

El frontend tiene dos ambientes (dev y produccion) y ambos se actualizaron. La diferencia clave es que se introdujo apiLocal derivado de api.

Compras-Frontend-Local/src/environments/environment.ts

```
// URL BASE del backend. NO la uses directo en un servicio: el backend esta
// dividido en dos prefijos (ver AGENTS.md / docs/PLAN.md seccion 1-2):
//
//   apiLocal -> `${api}/local` POS y gestion. Lo consume el EMPLEADO desde esta
//               app (Angular). Autenticado con JWT de empleado y autorizado por
//               permiso.
//   (futuro)  `${api}/tienda` Storefront. Lo consume el CLIENTE desde Next.js
//               (Compras-Frontend-Cliente). Nace en la Fase 3 del plan; el
//               catalogo sera publico y de solo lectura.
//
// Se deriva de `api` a proposito: cambiar de host (localhost -> Railway) se
// hace en UN solo lugar y ningun servicio se queda atras.
const api = 'http://localhost:8081/api';

export const environment = {
  production: false,
  api,
  apiLocal: `${api}/local`
};
```

El motivo de derivar apiLocal de api en vez de escribir las dos URLs enteras: la raiz api cambia entre dev y produccion. Si apiLocal estuviera escrito a mano, habria que cambiar DOS constantes por ambiente. Al derivarlo con template string, cambiar la raiz propaga el cambio.

El patron real y por que la autorreferencia no funciona

```
// FORMA INCORRECTA: autorreferencia
export const environment = {
  api: 'http://localhost:8081/api',
  apiLocal: `${environment.api}/local`,    // <-- environment aun no existe aqui
};

// FORMA CORRECTA: variable intermedia declarada antes
const api = 'http://localhost:8081/api';
export const environment = {
  api,
  apiLocal: `${api}/local`,
};
```

Trampa: la autorreferencia en un objeto exportado

No se puede escribir `apiLocal: `${environment.api}/local`` DENTRO del mismo objeto que se esta exportando: seria autodreferente y `environment` todavia no esta definido en ese punto. La variable intermedia `const api = ...` declarada antes resuelve el problema.

Los 12 servicios HTTP migrados

Los 12 servicios de la capa `core/service/` cambiaron su base de `environment.api` a `environment.apiLocal`. El unico cambio en cada uno es la constante de la URL:

Compras-Frontend-Local - sale-service.ts: la base se deriva de apiLocal

```
//Se usará en features/cobro/cobro.ts
export class SaleService {
  private api = `${environment.apiLocal}/sales`;
  private apiUrl = `${environment.apiLocal}/products`;

  /*private api = 'http://localhost:8081/api/local/sales';
  private apiUrl = 'http://localhost:8081/api/local/products';*/

  constructor(private http: HttpClient){}

  //Se procesa la venta, pero solo hasta que se realiza
  //El carro de ventas nunca de envía a la BD
  processSale(request: SaleRequest): Observable<SaleResponse>{
    return this.http.post<SaleResponse>(this.api, request);
  }
}
```

Regla para servicios futuros

Para cualquier servicio NUEVO hay que usar `${environment.apiLocal}` (con 'Local'), NUNCA `${environment.api}` directo. Escribir `${environment.api}/sales` funciona en desarrollo (no da error, solo pega en la ruta vieja sin prefijo) y revienta en produccion, o peor: pega contra un endpoint equivocado que alguien dejo ahi. El error no es visible hasta que cambia el entorno.

8. Tablas de base de datos, PK y FK: por que no cambio nada

Cero cambios de esquema

La Fase 1 no modifiko ninguna tabla, ninguna columna, ninguna clave primaria ni ninguna clave foranea. No se creo ninguna migracion nueva. Al cerrar la Fase 1 el proyecto tenia una sola migracion, V1__init.sql. (Despues, el punto 2 de esta serie agrego V2__venta_confirmada.sql: ese si cambio el esquema, y es el unico cambio de esquema del proyecto hasta la fecha.)

Esto merece justificacion, porque parece increible que un renombre de 164 archivos no toque el esquema. La respuesta es que las tablas ya se llaman por el DOMINIO (sales, products, purchases), no por el nombre del paquete Java. com.erikjarquin.compras.model.entity.SaleEntity mapea a la tabla public.sales mediante @Table(name = "sales"). El paquete Java y el nombre de la tabla son dos cosas sin relacion.

src/main/java/com/erikjarquin/compras/model/entity/SaleEntity.java (cabecera)

```
package com.erikjarquin.compras.model.entity;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;

import com.erikjarquin.compras.model.enums.PaymentMethod;
import com.erikjarquin.compras.model.enums.PaymentStatus;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
```

Tabla	Clave primaria	Claves forneas	Entidad JPA
sales	sales_pkey (id)	cash_register_id -> cash_registers(id)	SaleEntity
sale_details	sale_details_pkey (id)	sale_id -> sales(id), product_id -> products(id)	SaleDetailEntity
products	products_pkey (id)	category_id -> categories(id)	ProductEntity
purchases	purchases_pkey (id)	provider_id -> providers(id)	PurchaseEntity
purchase_details	purchase_details_pkey (id)	purchase_id -> purchases(id), product_id -> products(id)	PurchaseDetailEntity
permissions	permissions_pkey (id)	-	PermissionEntity
roles	roles_pkey (id)	-	RoleEntity
role_permissions	uk_role_permission (role_id, permission_id)	role_id -> roles(id), permission_id -> permissions(id)	RolePermissionEntity
users	users_pkey (id)	role_id -> roles(id)	UserEntity
cash_registers	cash_registers_pkey (id)	-	CashRegisterEntity
categories	categories_pkey (id)	-	CategoryEntity
providers	providers_pkey (id)	-	ProviderEntity
payments	payments_pkey (id)	sale_id -> sales(id)	PaymentEntity

Los nombres de constraint los genera pg_dump y son opacos por consecuencia: fk6nruj5m7ntwhc29etigqnlk0m no dice nada. Los que si importan y estan escritos a mano son: sales_pkey, uk_role_permission (clave primaria COMPUERTA de la tabla puente N:M) y los CHECK de enumeraciones.

Por que role_permissions es especial

La tabla puente role_permissions NO tiene id. Su clave primaria es compuesta: PRIMARY KEY (role_id, permission_id). Eso impide duplicados a nivel de BD (un rol no puede tener dos veces el mismo permiso) y es la razon por la que al asignar permisos hay que resolver por el par de claves y no por un id. Es tambien lo que hace que el INSERT de permisos de la migracion V2 (documento 2 de esta serie) sea un INSERT ... SELECT ... NOT EXISTS y no un INSERT normal.

9. Endpoints: inventario completo

12 controllers, 59 endpoints, todos bajo /api/local. El conteo aumento de 57 a 59 con el ciclo de vida de la venta (los dos PATCH), documentado en el segundo PDF de esta serie.

Modulo	Base	Permisos	Endpoints
Auth	/api/local/auth	publico	login, forgot/reset/change-password, me, authorities
Users	/api/local/users	VER/CREAR/EDITAR/ACTIVAR/DESACTIVAR_USUARIOS	CRUD + PATCH /{id}/active
Roles	/api/local/roles	VER/CREAR/EDITAR/ELIMINAR_ROLES	CRUD
Permissions	/api/local/permissions	VER_ROLES	GET all
Products	/api/local/products	VER/CREAR/EDITAR/ELIMINAR/DESACTIVAR/ACTIVAR_PRODUCTOS	CRUD multipart, /barcode/{bc}, /search, ?category=, /inactive, PATCHs
Categories	/api/local/categories	VER/CREAR/EDITAR/ELIMINAR_CATEGORIAS	CRUD
Sales	/api/local/sales	VER/CREAR/CONFIRMAR/CANCELAR_VENTAS	POST, GET, GET /{id}, PATCH /{id}/confirm, PATCH /{id}/cancel
Payments	/api/local/payments	PROCESAR_PAGOS	card, status/{tx}, retry/{id}, reverse/{id}
Purchases	/api/local/purchases	VER/CREAR/CANCELAR_COMPRAS	GET (todas, por proveedor, detalle), POST, DELETE (cancelar)
Providers	/api/local/providers	VER/CREAR/EDITAR/ELIMINAR_PROVEEDORES	CRUD
Cash	/api/local/cash	VER/ABRIR/CERRAR/CORTE_CAJA	open, close, summary, active
Reports	/api/local/reports	VER_REPORTES	trend, top-products, payment-methods, categories, low-stock, margins, summary

Los endpoints de /api/uploads quedan fuera de este conteo porque no pertenecen a ningun modulo de negocio: los sirve WebConfig directamente desde disco.

10. Glosario de anotaciones y conceptos

Todo lo que aparece en el código de esta fase, definido. La idea es que este documento sirva para replicar el trabajo, no solo para leerlo.

Anotacion / concepto	Que hace y cuando se usa
@RestController	Marca una clase como controlador REST. Equivale a @Controller + @ResponseBody: cada metodo devuelve el objeto Java, que Jackson serializa a JSON, en vez de resolver el nombre contra una plantilla HTML. Un controller de este proyecto devuelve JSON siempre.
@RequestMapping("/api/local/sales")	Prefijo de ruta comun a todos los metodos de la clase. Sin el, cada metodo tendria que declarar la ruta completa. Es el equivalente backend de un router del frontend.
@GetMapping / @PostMapping / @PatchMapping / @PutMapping / @DeleteMapping	Atajo de @RequestMapping(method = ...). Patch se usa cuando la operacion es un cambio PARCIAL: no se reenvia el recurso entero, solo se pide mover su estado. Es la semantica correcta para confirmar/anular y ademas evita que un PUT pise campos que otro usuario cambio.
@PathVariable Long id	Lee un segmento variable de la URL. En GET /api/local/sales/5, el 5 se inyecta en el parametro id. Sin @PathVariable Spring no sabe de donde sacarlo.
@RequestBody SaleRequest request	Deserializa el cuerpo JSON de la peticion al objeto. Jackson mapea cada clave JSON a la propiedad Java. Es lo que hace que el frontend mande un objeto tipado en vez de form-urlencoded.
@PreAuthorize("hasAuthority('CREAR_VENTAS')")	Spring Security method-level: antes de invocar el metodo evalua la expresion. hasAuthority() compara contra la lista de authorities del usuario autenticado. Si da false, la peticion se corta con 403 ANTES de tocar la base de datos. El string debe coincidir exactamente con el enum PermissionName.
ResponseEntity.ok(...)	Envoltorio que permite fijar codigo HTTP y headers. ok() es el atajo de 200. Existe porque un controller que devuelve el objeto crudo siempre responde 200, y a veces hace falta 201 Created o 204 No Content.
@Value("\${CORS_ALLOWED_ORIGINS:...}")	Inyecta un valor del archivo de configuracion. El texto tras los dos puntos es el valor por defecto, que se usa si la variable de entorno no esta definida. Sirve para externalizar lo que cambia entre entornos sin tocar codigo.
@Entity / @Table / @Id / @GeneratedValue / @Column	Mapeo objeto-relacional. @Entity marca la clase como tabla; @Table(name=...) indica el nombre real (si no, JPA derivaria 'sale_entity'); @Id marca la clave primaria; @GeneratedValue(IDENTITY) delega la generacion a la secuencia de la BD; @Column(nullable=false) documenta la restriccion NOT NULL.
@Transactional	Ejecuta el metodo dentro de una transaccion: todo se confirma o todo se revierte. Es lo que hace coherente un create() de venta que inserta cabecera, renglones y descuenta stock. rollbackFor = Exception.class hace que tambien revierta ante excepciones verificadas.
@RestControllerAdvice / @ExceptionHandler	Manejador global de excepciones. @RestControllerAdvice en la clase significa 'aplicate a todos los controllers'; @ExceptionHandler sobre un metodo indica que tipo de excepcion traduce. Es lo que convierte una SaleException de Java en un JSON {code, message} con el codigo HTTP correcto, sin try/catch en cada controller.
git mv	Equivale a mv + git add. Renombra un archivo y registra el cambio. La ventaja sobre mv es la deteccion de renombre en el historial.
MockMvc / @WebMvcTest / @MockitoBean / @WithMockUser	Testing web sin servidor: MockMvc simula la peticion HTTP completa contra el controller. @WebMvcTest carga solo la capa web, sin base de datos. @MockitoBean reemplaza un bean real por un doble de prueba. @WithMockUser establece el usuario autenticado con sus permisos, para poder probar el 403.

11. Verificacion: como se comprabo que nada se rompio

Un renombre de esta magnitud solo se puede dar por bueno con la suite de tests en verde. Este proyecto tiene 153 tests: 129 previos a este trabajo y 24 del ciclo de vida de la venta, documentado en el segundo PDF de la serie.

Los comandos de verificación

```
# Suite completa sin la BD real (recomendado en maquinas de desarrollo)
.\mvnw.cmd -o clean test "-Dtest=!ComprasApplicationTests" \
    "-DfailIfNoSpecifiedTests=false"

# Resultado: Tests run: 153, Failures: 0, Errors: 0, Skipped: 0
# BUILD SUCCESS

# Frontend
npx ng build # -> 'Application bundle generation complete'
```

Que cubre la suite	Como
Los 13 controllers	@WebMvcTest + MockMvc: se verifican las URLs nuevas.
Repositorios	@DataJpaTest sobre H2: consultas, EntityGraph, N+1.
Servicios	Mockito: reglas de negocio sin base de datos.
Almacenamiento de archivos	FileStorageServiceTest: extension, magic bytes, 5MB.
Bootstraps de arranque	RolePermissionBootstrapTest: siembra y no-resiembra.

Por que un test queda fuera

ComprasApplicationTests es el UNICO test anotado con @SpringBootTest y NO se puede ejecutar sin una base de datos real accesible. Falla con un error de autenticacion contra el pooler de Supabase configurado en el perfil local. Eso es preexistente, no un defecto de la Fase 1: por eso la suite se corre excluyendolo.

En el frontend, npx ng build debe terminar con 'Application bundle generation complete' y sin errores de compilacion de plantillas. Los warnings de CommonJS (canvg, html2canvas, jspdf) son preexistentes y benignos: vienen de las librerias de exportacion a PDF y Excel.

12. Trampas y errores que se cometieron en el camino

Un AGENTS.md desactualizado resucito un controller muerto

El archivo de contexto de otra sesion decia que existia un endpoint GET /ping. Otra maquina lo recreaciono siguiendo esas instrucciones. Se elimino de nuevo y se documento explicitamente que no debe recrearse. /ping no existe y no debe usarse como healthcheck de Railway.

El CHECK permissions_name_check no lo actualiza Hibernate

El enum PermissionName vive en Java, pero la lista de nombres permitidos esta tambien fijada en la base de datos por un CHECK. ddl-auto=update NO puede ampliar un CHECK (es DDL puro), asi que agregar un permiso sin actualizar la migracion hace que el arranque falle con 'violates check constraint'. Es la razon principal por la que se adopto Flyway.

pg_dump antepone directivas restrict y unrestrict

PostgreSQL 18 las emite en cada dump. Son comandos de psql, no SQL: Flyway las rechaza con 'error de sintaxis'. Si se regenera V1__init.sql a mano, hay que quitarlas.

application-local.yaml no esta en git

Al ser gitignored, en cada maquina hay que recrearlo. Sin el, la app no arranca: las variables \${DB_USER}, \${JWT_SECRET} y demas fallan de inmediato.

Los parentesis en la ruta rompen el runner de tests del frontend

La carpeta del proyecto se llama 'Sistema de ventas (comida)'. Los parentesis rompen el patron de busqueda de archivos de test de Vitest, y 'npm test' responde 'No test files found' aunque existan specs. Es un problema del entorno, no del codigo.

Autorreferencia en un objeto exportado de TypeScript

Escribir apiLocal: ``${environment.api}/local`` dentro del mismo objeto environment que se exporta no funciona: environment aun no existe. Resolver con una variable intermedia declarada antes.